IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

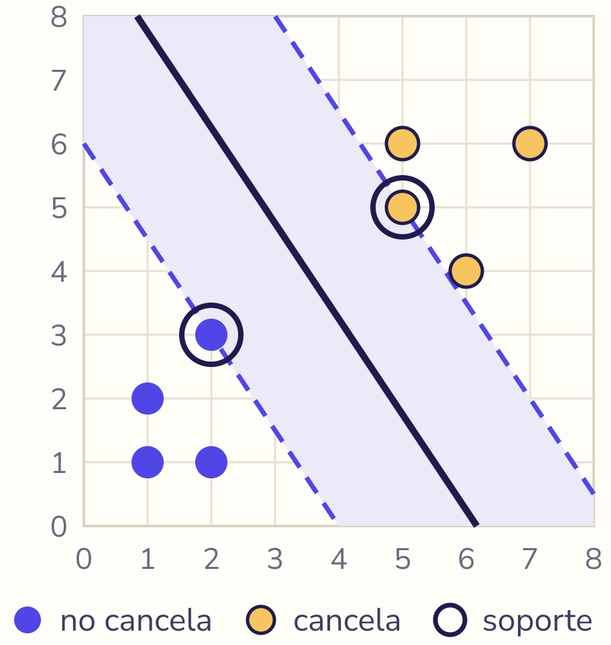
Máquinas de vectores de soporte (SVM)

La calle más ancha entre dos grupos, y cuándo no merece la pena

Sprint 8 · S08_04 · 40 min en casa (ejercicios 1 a 4 y mini-reto) · 10 min de teoría el 12/11

IDEA · EN CLASE 12/11

La calle más ancha entre dos grupos



Frontera

La línea del centro de la calle: a un lado se predice una clase, al otro, la otra.

Margen

El ancho de la calle, aquí **3,61**. La SVM busca la calle **más ancha** posible.

Vectores de soporte

Los puntos que tocan las aceras, aquí **(2, 3)** y **(5, 5)**: ellos sujetan la calle.

IDEA · EN CLASE 12/11

Solo mandan los vectores de soporte

Qué hago con los 8 puntos	¿Cambia la recta?
Quito uno de los 6 puntos lejanos	**No**: las 6 pruebas dan la misma recta
Muevo el punto (2, 3) a (1, 3)	**Sí**: de 6x + 4y = 37 pasa a 2x + y = 10
Quito el punto (2, 3)	**Sí**: cambia la recta y ahora hay 4 vectores de soporte

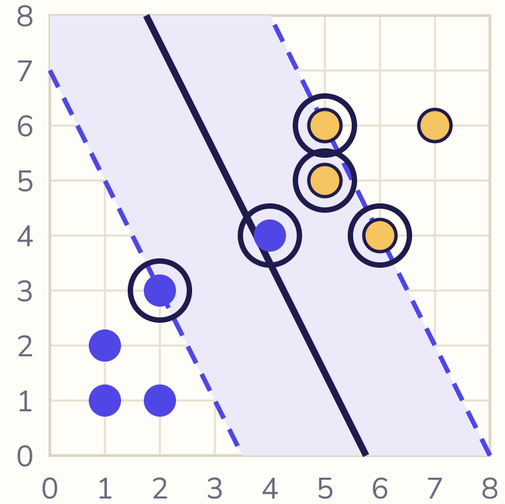
```
svm = SVC(kernel="linear", C=1000).fit(A, y)
svm.support_vectors_      # [[2, 3], [5, 5]]
svm.n_support_            # [1, 1] (uno por clase)
```

Como en una tienda de campaña: solo los palos que la sujetan importan, el resto de la tela no.

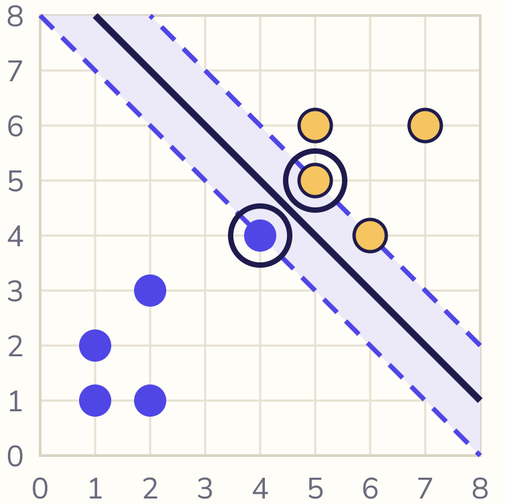
IDEA · EN CLASE 12/11

C: cuánto cuesta equivocarse

C = 0,1 · calle ancha



C = 100 · calle estrecha



C = 0,1 (pequeño)

Margen **4,03** · 1 error en train · 5 vectores de soporte

C = 100 (grande)

Margen **1,41** · 0 errores en train · 2 vectores de soporte

C grande: calle estrecha, ajusta mucho (riesgo de sobreajuste). C pequeño: calle ancha, tolera errores.

IDEA · EN CLASE 12/11

Cuando una recta no basta: el kernel

x	−4	−3	−2	−1	0	1	2	3	4
x^2	16	9	4	1	0	1	4	9	16
clase	0	0	0	1	1	1	0	0	0

```
x2 = np.c_[x, x**2]                                  # una columna nueva: x²
SVC(kernel="linear").fit(x, clase).score(x, clase)   # 0.67 (una recta no puede)
SVC(kernel="linear").fit(x2, clase).score(x2, clase) # 1.00 (con la columna x²)
SVC(kernel="rbf").fit(x, clase).score(x, clase)      # 1.00 (sin crear nada)
```

Kernel = curvar el espacio. El RBF lo hace solo, sin construir columnas, y deja fronteras curvas.

KERNEL RBF · EJERCICIO 1 (CASA)

gamma: hasta dónde llega cada punto

gamma	Exactitud en train	Exactitud en test	Puntos guardados	Qué pasa
0,1	0,80	0,77	201	**Infraajusta**: frontera casi recta
1	0,85	0,84	104	Equilibrio
100	0,96	0,74	196	**Sobreajusta**: islotes alrededor de cada punto

```
mod = SVC(kernel="rbf", C=1, gamma=100).fit(X_train, y_train)
mod.score(X_train, y_train), mod.score(X_test, y_test)    # (0.96, 0.74)
mod.n_support_.sum()                                      # 196 de 210: casi todos
```

gamma bajo: influencia lejana, frontera suave. gamma alto: influencia local, frontera retorcida.

DATOS DE TURISDATA · EJERCICIO 2 (CASA)

Escalar es obligatorio: la SVM mide distancias

Reserva	precio_noche	noches
A	80	2
B	90	7
C	200	2

Pareja	Distancia sin escalar	Escalando
A – B	11,18	2,13
A – C	120,00	2,21
B – C	110,11	2,93

```
pipe = Pipeline([("prep", prep_escalado()),
                 ("svm", SVC(kernel="rbf", C=1))])
auc = roc_auc_score(y_test, pipe.decision_function(X_test))
```

Sin escalar, A y B parecen casi iguales pese a 5 noches de diferencia: manda el precio, que es la cifra más grande.

AJUSTE · EJERCICIO 3 (CASA)

Probar C y gamma con la rejilla

C \ gamma	0,1	1
0,1	0,795	0,840
1	0,835	**0,860**
10	0,835	0,825

6 combinaciones × 5 pliegues

= **30 ajustes**, y uno más al final con la mejor combinación.

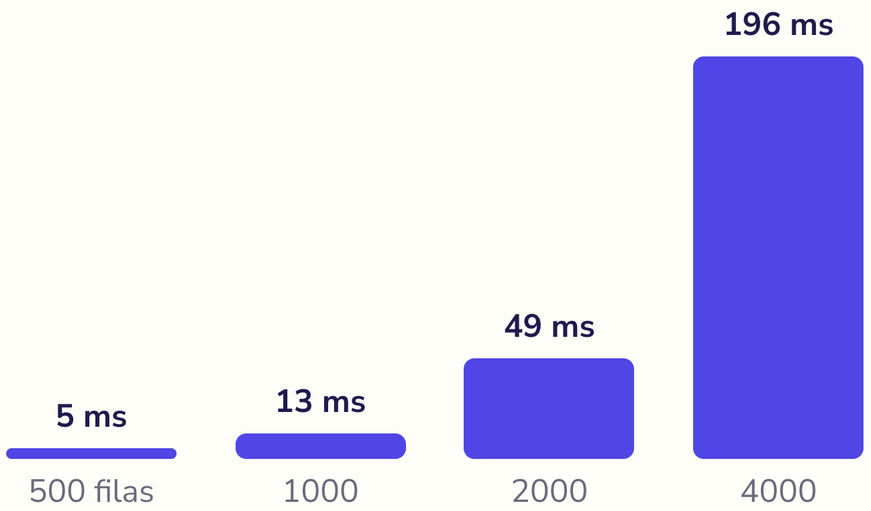
Cuidado con los empates

Las cuatro mejores están a solo 0,025; entre pliegues, la nota varía unos 0,06.

```
rejilla = {"C": [0.1, 1, 10], "gamma": [0.1, 1]}
busqueda = GridSearchCV(SVC(kernel="rbf"), rejilla, cv=5)
busqueda.fit(X, y); busqueda.best_params_    # {'C': 1, 'gamma': 1}
```

COSTE Y LÍMITES · EJERCICIO 4 (CASA)

Con muchas filas, la SVM se hace lenta



×4 filas, ×15 tiempo

De 1000 a 4000 filas, el tiempo se multiplicó por **14 a 21** en siete pruebas: crece casi como el cuadrado de las filas.

predict_proba pide **probability=True**

Da probabilidades, pero entrena unas **5 veces** más despacio. Para ordenar basta **decision_function**.

¿5 millones de reservas? Mejor LinearSVC o un boosting: la SVM RBF no escala.

MINI-RETO (CASA)

Tres SVM frente a frente

```
variantes = {
  "lineal": SVC(kernel="linear", C=1),
  "rbf_defecto": SVC(kernel="rbf", C=1),
  "rbf_ajustada": SVC(kernel="rbf",
      C=C_aj, gamma=g_aj)}
for nombre, svm in variantes.items():
  pipe = Pipeline([("prep", prep_escalado()),
                   ("svm", svm)]).fit(...)
```

Juguete (420 train)	auc_test	n_soporte	% del train
lineal	0,928	176	42 %
rbf_defecto	0,929	167	40 %
rbf_ajustada	0,935	170	40 %

tabla_svm: filas **lineal**, **rbf_defecto** y **rbf_ajustada**; columnas **auc_test**, **n_soporte** y **segundos_ajuste**. Los **mejores_params** llevan el prefijo **svm__**: **quítalo**.

Mira dos cosas: ¿gana la RBF a la lineal? ¿Qué parte del train guarda cada modelo?

HOY HAS APRENDIDO

Margen, curvas y coste

La idea

La calle más ancha; solo los **vectores de soporte** la definen; **C** marca la tolerancia a errores

Las curvas

Kernel **RBF** y **gamma**; escalar es obligatorio; **GridSearchCV** ajusta **C** y **gamma**

Los límites

Lenta con muchas filas, poco explicable y sin probabilidades de serie

Siguiente: S08_05 · buscar hiperparámetros (guía del paso 3 del reto)